

GIORNO 07

Liste, dizionari e tuple: dove si mettono i dati quando sono tanti

Cosa saprete fare stasera

Mettere molti valori in una lista, prenderli per posizione, aggiungerne e toglierne

Scorrere una collezione con un for, sapendo anche a che posizione siete

Tenere insieme un nome e un valore in un dizionario, e trovare il valore senza cercarlo

Contare quante volte compare ogni cosa, con quattro parole di codice per sempre

Costruire una tabella come lista di tuple e ricavarne totali, medie e classifiche

Scegliere, davanti a un dato nuovo, quale delle tre strutture serve, e dire perché

Tre strutture in una giornata sono tante: è la giornata più carica del blocco.

In compenso quasi tutto quello che viene dopo è costruito sopra queste tre cose.

1. RIPASSO E DOMANDE

return contro print, e i cinque prerequisiti della giornata

1.1 return contro print: che cosa resta in mano a chi chiama

```
# 1. return CONSEGNA il valore a chi ha chiamato, e chiude la funzione.

#   L'espressione calcola_totale(10.0, 3) DIVENTA quel valore.

def calcola_totale(prezzo, quantita):

    # Restituisce il totale di riga

    return prezzo * quantita


# 2. Il valore consegnato si cattura in una variabile: da qui in poi e' un

#   float come un altro, riusabile quante volte serve.

risultato = calcola_totale(10.0, 3)

print("Il risultato vale:", risultato)


# TRABOCCHETTO: e' questa riga a distinguere i due casi, non quella sopra.

#   con return -> risultato vale 30.0 (float), e si puo' raddoppiare

#   con print   -> risultato vale None, e non c'e' niente da raddoppiare

print("Il doppio vale:", risultato * 2)


# Il risultato vale: 30.0

# Il doppio vale: 60.0
```

1.2 Prerequisiti operativi: f-string, slicing, if, for, return

Dal Giorno	Che cosa serve oggi
02	f-string con larghezza e allineamento: tutti i report di oggi sono colonne
02	Lo slicing sulle stringhe, s[0:6]: oggi la stessa cosa sulle liste
04	if / elif / else e in come test su una stringa
05	for, contatori e accumulatori: oggi il for gira su collezioni vere
06	Funzioni con parametri e return, anche di due valori

Le strutture di oggi sono nuove, il resto e' roba vostra. Se una di queste cinque vi sembra sfocata, recuperatela adesso: nel pomeriggio la userete piu' volte per esercizio.

2. DALLE VARIABILI SINGOLE ALLE COLLEZIONI

Il numero di variabili si decide scrivendo, i dati arrivano dopo

2.1 Il limite della variabile singola: il numero si fissa scrivendo

Il totale delle vendite di sei mesi, con quello che sapevate ieri sera:
sei variabili e una somma lunga tre righe. Funziona.

Poi arrivano ottobre e novembre, e dovete toccare DUE punti del programma.
Poi il capo chiede il mese peggiore, e serve una catena di if a sei rami.
E a quel punto il programma non lo tocca più nessuno.

Il problema non è la fatica.

È che il numero di variabili è deciso QUANDO SCRIVETE IL CODICE,
e i dati veri arrivano dopo.

Domanda che smonta tutto: quante cose devo conservare?

Se la risposta è "dipende", non serve una variabile per ciascuna.

2.2 La lista: un nome solo per molti valori

```
# 1. Un nome, sei valori. Il tipo e' list: contenitore ordinato, che sa
#     quanti elementi ha e li tiene nell'ordine in cui li avete scritti.
vendite = [130, 95, 140, 85, 160, 110]

# 2. Quattro funzioni GENERALI: lavorano su una collezione qualsiasi, e
#     nessuna di loro nomina il numero sei.

print("Mesi rilevati:", len(vendite))
print("Totale pezzi:", sum(vendite))
print("Mese peggiore:", min(vendite))
print("Mese migliore:", max(vendite))

# TRABOCCHETTO: min() e max() rispondono QUANTO, mai QUALE mese.
#     Il nome del mese peggiore si ricava in un secondo passo: vedi 9.2.

# Mesi rilevati: 6
# Totale pezzi: 720
# Mese peggiore: 85
# Mese migliore: 160
```

2.3 Lista, dizionario e tupla: tre contenitori a confronto

Struttura	Si scrive	Serve quando	Esempio di oggi
Lista	[...]	conta l'ordine, gli elementi cambiano	la coda d'attesa, il catalogo
Dizionario	{chiave: valore}	conta il nome, non la posizione	il listino per zona, i conteggi
Tupla	(...)	una riga di dati che non deve cambiare	una candidatura, un ordine

Tre contenitori, tre mestieri. La scelta sbagliata non da' errore: si paga fra sei mesi, quando qualcuno deve modificare il programma. E' la tabella da tenere aperta tutto il giorno, e torna al capitolo 15 con tre righe in piu'.

2.3 Tre parentesi, tre tipi

```
# Tre parentesi diverse, tre tipi diversi. La parentesi e' la dichiarazione
# di intenti: chi legge il codice capisce il mestiere prima del contenuto.
#   quadre [ ] -> list    elenco ordinato, si allunga e si accorcia
#   graffe { } -> dict    coppie chiave-valore, si cerca per nome
#   tonde  ( ) -> tuple   campi fissi di una cosa sola, non si toccano

lista = ["NORD", "CENTRO", "SUD"]

dizionario = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50}

tupla = ("LS-2026-MI-0042", "NORD", 12.5)


# TRABOCCHETTO: le tre parentesi qui sotto sembrano la stessa cosa e non lo
# sono. Nella lista e nella tupla il numero 1 e' una POSIZIONE; nel
# dizionario "CENTRO" e' una CHIAVE, e non ha nessuna posizione.

print(lista[1], dizionario["CENTRO"], tupla[1])
```

Il prezzo esce come 7.0 e non 7.00: il valore salvato e' un float, la forma con due decimali e' una scelta di stampa e si chiede con :.2f. E' il Giorno 02 che torna.

```
# CENTRO 7.0 NORD
```

3. LA LISTA

Costruirla, guardarci dentro, e contare da zero

3.1 Costruire una lista e leggerla per indice

```
# 1. Costante, quindi maiuscola. La quadra aperta tiene insieme le righe.
CATALOGO = ["Cuffie Bluetooth", "Tastiera meccanica", "Mouse verticale",
            "Monitor 27 pollici", "Webcam HD", "Hub USB-C"]

# 2. len() conta gli ELEMENTI, non i caratteri: qui risponde sei, e non gli
#     interessa quanto sono lunghi i nomi dentro.
print("Quanti prodotti:", len(CATALOGO))

# 3. Le quadre dopo il nome sono l'ACCESSO PER INDICE: restituiscono un
#     elemento, non una lista.
print("Il primo:", CATALOGO[0])
print("Il terzo:", CATALOGO[2])

# 4. Indice negativo: si conta dal fondo, e -1 e' l'ultimo. Non esiste -0.
print("L'ultimo:", CATALOGO[-1])
print("Il penultimo:", CATALOGO[-2])

# TRABOCCHETTO: "il terzo" per un umano e' CATALOGO[2] per Python.
#     Nel nome della variabile dite sempre quale dei due state usando.

# Quanti prodotti: 6     Il primo: Cuffie Bluetooth
# Il terzo: Mouse verticale     L'ultimo: Hub USB-C
# Il penultimo: Webcam HD
```

3.2 L'indice parte da zero: posizioni positive e negative

Elemento	Indice positivo	Indice negativo
Cuffie Bluetooth	0	-6
Tastiera meccanica	1	-5
Mouse verticale	2	-4
Monitor 27 pollici	3	-3
Webcam HD	4	-2
Hub USB-C	5	-1

Va contato a schermo, non spiegato a parole. Sei elementi, indici da 0 a 5: l'indice piu' grande e' sempre len meno uno. E' la logica del piano terra, che sta al livello della strada e non e' il primo piano. Sondaggio in chat prima di proseguire: quanti elementi ha una lista il cui ultimo indice e' 4?

3.3 Assegnazione per posizione: lista[i] = valore

```
COLORI_TRIAGE = ["BIANCO", "VERDE", "GIALLO", "ROSSO"]
```

```
# Le stesse quadre, ma a SINISTRA dell'uguale: non chiedono l'elemento, lo
```

```
# sostituiscono. La lista resta lunga quattro: uno esce, uno entra.
```

```
COLORI_TRIAGE[1] = "AZZURRO"
```

```
# TRABOCCHETTO: qui sopra non nasce nessuna lista nuova. La lista e' la
```

```
# stessa di prima, cambiata da dentro: si dice che e' MUTABILE, ed e'
```

```
# la differenza che al capitolo 13 separa la lista dalla tupla.
```

```
print(COLORI_TRIAGE)
```

```
# ['BIANCO', 'AZZURRO', 'GIALLO', 'ROSSO']
```

Stampata così, la lista esce con le quadre e gli apici: e' come Python ve la mostra, non un formato per il cliente. Per un report si scorre e si stampa una riga per volta, al capitolo 6. Ricordatevi la riga con l'uguale: al capitolo 13 vedremo che con una tupla non si può fare, ed e' il motivo per cui la tupla esiste.

3.4 Da provare subito: len(), indice 0 e indice -1

```
zone = ["NORD", "CENTRO", "SUD", "ISOLE"]

# Quattro elementi, indici da 0 a 3. L'indice piu' grande e' len() meno uno:
# scrivetelo sul quaderno, perche' e' meta' degli IndexError della giornata.
print(len(zone), zone[0], zone[-1])

print(zone[1], "sta in posizione 1 ed è il secondo")

# 4 NORD ISOLE

# CENTRO sta in posizione 1 ed è il secondo
```

Tre righe da digitare adesso, contando a voce mentre scrivete: zero, uno, due. Se avete scritto zone[4] per curiosita', avete appena visto il vostro primo IndexError. E' il momento giusto per vederlo.

4. MODIFICARE UNA LISTA

Quattro verbi, e l'uguale di troppo

4.1 append, insert, remove, pop: i metodi che modificano

Metodo	Che cosa fa	Restituisce qualcosa?
append(x)	aggiunge x in fondo	no
insert(i, x)	infilare x alla posizione i, spostando gli altri	no
remove(x)	toglie la prima occorrenza del valore x	no
pop() / pop(i)	toglie l'ultimo, o quello in posizione i	sì, l'elemento tolto

Una lista non è scolpita: si allunga, si accorcia, si riordina. Questi quattro metodi coprono il novantacinque per cento dei casi. L'ultima colonna è la più importante: append, insert e remove lavorano sulla lista e basta; pop vi mette in mano l'elemento tolto, e se non lo salvate lo avete buttato.

4.1 La coda d'attesa dell'Aurora: i quattro verbi in fila

```
# La coda d'attesa di uno sportello. Nome minuscolo: questa lista cambia.
attesa = ["Rossi Anna", "Bianchi Marco", "Verdi Luca"]

# 1. append(x): x va IN FONDO, sempre. E' il metodo che userete di piu',
#     perche' e' come si riempie una lista dentro un ciclo.
attesa.append("Neri Sara")

# 2. insert(i, x): x va ALLA POSIZIONE i, e tutti gli altri scalano di uno.
#     insert(0, x) mette in testa: e' il sorpasso per urgenza.
attesa.insert(0, "Gallo Pietro")

# 3. remove(x): toglie la PRIMA occorrenza del VALORE x. Se x non c'e',
#     solleva ValueError e il programma si ferma: vedi 4.2.
attesa.remove("Bianchi Marco")

# 4. pop(i): toglie l'elemento IN POSIZIONE i e ve lo RESTITUISCE. E' l'unico
#     dei quattro che va scritto a destra di un uguale.
chiamato = attesa.pop(0)
print("Chiamato allo sportello:", chiamato)
print("Dopo pop:", attesa)

# TRABOCCHETTO: tre righe su quattro qui sopra NON hanno l'uguale.
#     attesa.append("Neri Sara")          corretto: modifica e basta
#     attesa = attesa.append("Neri Sara") distrugge la lista: vedi 4.3
```

4.2 remove toglie per valore, pop per posizione

```
zone = ["NORD", "CENTRO", "SUD", "ISOLE"]

zone.remove("SUD")      # per NOME: "togli quella che si chiama SUD"
zone.pop(0)             # per POSIZIONE: "togli quella che sta al posto 0"

# TRABOCCHETTO: le due righe si INFLUENZANO. Dopo remove("SUD") la lista e'
#   lunga tre e ISOLE e' scalato da 3 a 2. Se scrivete pop(3) invece di
#   pop(0), quella posizione non esiste piu': IndexError.

print(zone)

# ['CENTRO', 'ISOLE']
```

E' la confusione numero uno del capitolo. Come al guardaroba: potete dire "il cappotto blu" oppure "il numero 12". Due richieste diverse, due sportelli diversi. remove toglie solo la PRIMA occorrenza, e se il valore non c'e' si ferma con un ValueError.

4.3 lista = lista.append(x): il metodo che restituisce None

```
prodotti = ["Webcam HD", "Hub USB-C", "Cuffie Bluetooth"]

# append fa DUE cose: modifica la lista (e funziona) e restituisce None (ed
# e' il problema). L'uguale butta via la lista modificata e mette None al
# suo posto, sotto lo stesso nome: da qui in poi prodotti non e' una lista.
prodotti = prodotti.append("Mouse verticale")

print(prodotti)

# None

prodotti = ["Webcam HD", "Hub USB-C", "Cuffie Bluetooth"]

# La forma giusta: il metodo sta da solo sulla riga, senza uguale davanti.
prodotti.append("Mouse verticale")

# TRABOCCHETTO: il criterio non e' "metto l'uguale a caso".
#   MODIFICANO    -> append, insert, remove, sort, reverse: niente uguale
#   COSTRUISCONO -> sorted(), lista[:]: si assegnano, o il risultato
#
#               nuovo non lo tiene nessuno

print(prodotti)

# ['Webcam HD', 'Hub USB-C', 'Cuffie Bluetooth', 'Mouse verticale']
```

5. CERCARE E CONTARE

`in`, `index`, `count`: e uno solo dei tre perdona

5.1 L'operatore in: appartenenza, non sottostringa

```
CATALOGO = ["Cuffie Bluetooth", "Tastiera meccanica", "Mouse verticale"]
```

```
# in su una lista e' un test di APPARTENENZA: restituisce un bool, e il
```

```
# posto giusto dove metterlo e' la condizione di un if.
```

```
print("Mouse verticale" in CATALOGO)
```

```
print("Webcam HD" in CATALOGO)
```

```
# TRABOCCHETTO: su una LISTA l'elemento deve combaciare per intero.
```

```
# "Mouse" in "Mouse verticale" -> True (stringa: sottostringa)
```

```
# "Mouse" in CATALOGO -> False (lista: elemento intero)
```

```
print("Mouse" in CATALOGO)
```

```
# True
```

```
# False
```

```
# False
```

5.2 index(): la posizione, e il controllo con in che viene prima

```
def posizione_o_zero(elenco, cercato):  
    # Posizione umana dell'elemento, oppure 0 se non c'e'  
  
    # La guardia: index() e' l'operazione pericolosa, in non fallisce mai.  
  
    if cercato in elenco:  
  
        # + 1 converte la posizione interna (da 0) in quella che si dice ad  
  
        # alta voce (da 1). Due numeri diversi per la stessa cosa.  
  
        return elenco.index(cercato) + 1  
  
    # Ripiego: 0 non e' una posizione umana valida, quindi vale "non trovato"  
  
    # senza ambiguita'.  
  
    return 0
```

```
zone = ["NORD", "CENTRO", "SUD"]  
print(posizione_o_zero(zone, "SUD"))
```

```
# TRABOCCHETTO: senza l'if, questa riga fermerebbe il programma con  
  
# ValueError: 'ISOLE' is not in list. Restituire 0 e' una DECISIONE di  
  
# progetto, ed e' la stessa che prenderemo con .get() al capitolo 11.  
print(posizione_o_zero(zone, "ISOLE"))
```

```
# 3
```

```
# 0
```

5.3 count(): quante occorrenze, e perché non si ferma mai

```
triage = ["VERDE", "GIALLO", "VERDE", "ROSSO", "VERDE", "BIANCO"]

# count(x) scorre tutta la lista e conta le occorrenze di x. Su un valore
# assente risponde 0: e' l'unico dei tre che non solleva nessuna eccezione.
# TRABOCCHETTO: risponde 0 anche se la lista e' quella sbagliata o il valore
# e' scritto male. Uno zero da count non distingue "non c'e'" da "ho
# cercato la cosa sbagliata".

print(triage.count("VERDE"), triage.count("ROSSO"), triage.count("NERO"))

# 3 1 0
```

6. SCORRERE UNA LISTA

Il for come lo volevate nel Giorno 05

6.1 for elemento in lista: iterazione diretta, senza indice

```
CATALOGO = ["Cuffie Bluetooth", "Tastiera meccanica", "Mouse verticale"]

# ITERAZIONE DIRETTA: a destra del in c'e' la collezione, a sinistra il nome
# che a ogni giro vale l'elemento successivo. Nessun indice, nessun range(),
# nessun len(): tre occasioni di sbagliare tolte di mezzo.
for prodotto in CATALOGO:
    # prodotto e' una str, e vale un elemento alla volta.
    print("Prodotto a listino:", prodotto)

# Prodotto a listino: Cuffie Bluetooth
# Prodotto a listino: Tastiera meccanica
# Prodotto a listino: Mouse verticale
```

Nel Giorno 05 avete scorso stringhe e range, e vi avevamo detto che sulle liste il for avrebbe dato il meglio. Eccoci: e' la forma da preferire SEMPRE, quando la posizione non vi serve.

6.2 Accumulatore e contatore: inizializzazione fuori dal ciclo

```
PEZZI = [130, 95, 140, 85, 160, 110]

# Due variabili, due mestieri, azzerate QUI: fuori dal ciclo, una volta sola.
#   totale      ACCUMULATORE -> somma i valori, cresce di venduti
#   sopra_cento CONTATORE    -> conta i casi, cresce sempre di uno
totale = 0
sopra_cento = 0

for venduti in PEZZI:
    totale += venduti

    # Il contatore sta dentro un if: conta solo i giri che superano il
    # filtro. L'accumulatore no: quello somma tutto.

    if venduti > 100:
        sopra_cento += 1

# TRABOCCHETTO: spostate totale = 0 dentro il for e il programma non sbaglia
# di poco, sbaglia di tutto: si riazzerà a ogni giro e il "totale" finale
# è l'ultimo elemento, 110. Nessun errore a schermo.
print("Totale:", totale)

print("Mesi sopra i cento pezzi:", sopra_cento)

# Totale: 720
# Mesi sopra i cento pezzi: 4
```

6.3 enumerate(): indice ed elemento nello stesso giro

```
CATALOGO = ["Cuffie Bluetooth", "Tastiera meccanica", "Mouse verticale"]

# DUE nomi a sinistra del in, perche' enumerate consegna a ogni giro una
# COPPIA (indice, elemento). E' lo stesso unpacking del Giorno 03.
for posizione, prodotto in enumerate(CATALOGO):
    # posizione e' int, prodotto e' str: due tipi nello stesso giro.
    print(posizione, prodotto)

# 0 Cuffie Bluetooth
# 1 Tastiera meccanica
# 2 Mouse verticale
```

Per un elenco numerato servono due cose a ogni giro: l'elemento e il numero di riga. Si puo' fare con un contatore a mano, ed e' una riga di troppo. enumerate() fa la stessa cosa e mette in chiaro l'intenzione. Parte da zero, perche' e' la posizione vera.

6.3 enumerate(CATALOGO, 1): il numero di partenza

```
CATALOGO = ["Cuffie Bluetooth", "Tastiera meccanica", "Mouse verticale"]

# Il secondo argomento di enumerate e' il NUMERO DI PARTENZA del contatore.
# TRABOCCHETTO: non sposta gli elementi e non salta il primo. Cambia solo il
#   numero stampato accanto: legge sempre tutti e tre i prodotti.
for posizione, prodotto in enumerate(CATALOGO, 1):
    print(f"   {posizione}. {prodotto}")

#   1. Cuffie Bluetooth
#   2. Tastiera meccanica
#   3. Mouse verticale
```

Se volete il numero da leggere ad alta voce, glielo dite. Vi risparmia il + 1 sparso in cinque punti, e il + 1 dimenticato in uno dei cinque.

6.4 Due liste parallele: stesso indice, e perché è fragile

```
# Due LISTE PARALLELE: l'accoppiamento non e' scritto da nessuna parte, sta
# solo nel fatto che la posizione 0 dell'una va con la posizione 0 dell'altra.

MESI = ["Aprile", "Maggio", "Giugno"]
PEZZI = [130, 95, 140]

# Si scorre UNA lista sola e si usa il suo indice per pescare nell'altra.
for posizione, mese in enumerate(MESI):

    # :<10 a sinistra su dieci colonne, :>4 a destra su quattro: e' la
    # stessa f-string del Giorno 02, ed e' cio' che tiene le colonne.
    print(f" {mese:<10} {PEZZI[posizione]:>4}")

# TRABOCCHETTO: se PEZZI ha un elemento in meno, IndexError alla riga del
# print; se ne ha uno IN PIU', il ciclo lo ignora in silenzio. Il secondo
# caso e' il pericoloso.

#  Aprile      130
#  Maggio      95
#  Giugno      140
```

7. FETTE DI LISTA

Lo slicing del Giorno 02, sulle liste

7.1 Lo slicing lista[a:b]: estremo destro escluso

```
MESSAGGI = ["m1", "m2", "m3", "m4", "m5", "m6", "m7", "m8", "m9", "m10"]
```

```
# 1. Lo slicing restituisce SEMPRE una lista nuova, anche di un elemento
# solo; MESSAGGI[0] restituisce l'elemento. Due operazioni diverse
# scritte con le stesse quadre.
```

```
print(MESSAGGI[0:3])
```

```
# 2. Estremo mancante = "fino in fondo" a destra, "dall'inizio" a sinistra.
```

```
print(MESSAGGI[7:])
```

```
print(MESSAGGI[:2])
```

```
# 3. [-2:] sono gli ultimi due, qualunque sia la lunghezza della lista.
```

```
print(MESSAGGI[-2:])
```

```
# TRABOCCHETTO: [0:3] da' TRE elementi, non quattro. L'estremo destro dice
```

```
# dove FERMARSI, e il vantaggio e' che [0:3] e [3:] si incastrano senza
```

```
# sovrapporsi: vedi 7.2.
```

```
# ['m1', 'm2', 'm3']
```

```
# ['m8', 'm9', 'm10']
```

```
# ['m1', 'm2']
```

```
# ['m9', 'm10']
```

7.1 Le cinque scritture da riconoscere

Scrittura	Vuol dire
<code>lista[0:3]</code>	i primi tre, cioè le posizioni 0, 1, 2
<code>lista[3:]</code>	dalla posizione 3 fino alla fine
<code>lista[:3]</code>	dall'inizio fino alla posizione 3 esclusa
<code>lista[-2:]</code>	gli ultimi due
<code>lista[:]</code>	tutta, ma in una lista NUOVA

L'ultima riga sembra inutile e non lo è: è la copia vera, e ci arriviamo al 7.3. Nelle soluzioni di oggi compare all'inizio di ogni `main()`, per non toccare la costante.

7.2 Partizionare una lista in due fette senza sovrapposizioni

```
ESEMPI = ["e1", "e2", "e3", "e4", "e5", "e6", "e7", "e8", "e9", "e10"]

# Il punto di taglio ha un nome ed e' scritto una volta sola: se cambia, si
# tocca QUI e le due fette restano coerenti da sole.
QUANTI_DI_STUDIO = 7

# La prima fetta arriva fino al taglio, escluso: e1..e7. La seconda parte
# dal taglio, incluso: e8..e10. Lo stesso numero usato due volte, ed e'
# l'estremo escluso a evitare la sovrapposizione.

studio = ESEMPI[0:QUANTI_DI_STUDIO]
esame = ESEMPI[QUANTI_DI_STUDIO:]

# TRABOCCHETTO: la tentazione e' scrivere [QUANTI_DI_STUDIO + 1:] per "non
# ripetere l'elemento". Quel + 1 fa sparire e8 da tutte e due le fette:
# len(studio) + len(esame) fa 9, non 10. Quella somma e' il controllo da
# fare ogni volta che si partiziona.

print("Studio:", len(studio), studio)
print("Esame: ", len(esame), esame)

# Studio: 7 ['e1', 'e2', 'e3', 'e4', 'e5', 'e6', 'e7']
# Esame:  3 ['e8', 'e9', 'e10']
```

7.3 Assegnazione contro copia: due nomi per lo stesso oggetto

```
listino_a = ["NORD", "CENTRO", "SUD"]

# Questo uguale NON copia niente: copia il riferimento, e da adesso due nomi
# diversi indicano lo stesso identico oggetto in memoria.

listino_b = listino_a

# append modifica l'oggetto DA DENTRO, e l'oggetto e' uno solo: la modifica
# si vede da tutti e due i nomi.

listino_b.append("ISOLE")

# TRABOCCHETTO: con numeri e stringhe non capita mai, perche' sono
# immutabili e ogni modifica crea un valore nuovo. Capita con liste e
# dizionari, che si cambiano senza crearne di nuovi.

print("listino_b:", listino_b)
print("listino_a:", listino_a)

# listino_b: ['NORD', 'CENTRO', 'SUD', 'ISOLE']
# listino_a: ['NORD', 'CENTRO', 'SUD', 'ISOLE']
```

7.3 La copia vera si chiede con lo slicing completo

```
listino_a = ["NORD", "CENTRO", "SUD"]

# Le due quadre vuote sono una FETTA COMPLETA, e lo slicing costruisce
# sempre una lista nuova: qui gli oggetti diventano due.

listino_b = listino_a[:]

listino_b.append("ISOLE")

# TRABOCCHETTO: il contenuto e' lo stesso, l'oggetto no. Serve per ordinare,
# tagliare o svuotare senza rovinare i dati di partenza: nelle soluzioni
# di oggi la trovate in cima a ogni main(), e non e' pignoleria.

print("listino_b:", listino_b)
print("listino_a:", listino_a)

# listino_b: ['NORD', 'CENTRO', 'SUD', 'ISOLE']
# listino_a: ['NORD', 'CENTRO', 'SUD']
```

7.4 Da provare subito: la lunghezza di una fetta è $b - a$

```
numeri = [10, 20, 30, 40, 50]
```

```
# Tre fette in una riga. Notate che numeri[-1:] e' una LISTA di un elemento,  
# [50], mentre numeri[-1] sarebbe l'intero 50: i due punti cambiano il tipo.  
print(numeri[1:4], numeri[:2], numeri[-1:])
```

```
# TRABOCCHETTO: 1:4 sembra "dalla 1 alla 4" e sono TRE elementi,  $4 - 1$ . La  
# lunghezza di una fetta e' la differenza fra i due numeri, ed e' il conto  
# da fare a mente prima di stampare, non dopo.  
print(len(numeri[1:4]), "elementi, dalla posizione 1 alla 3")
```

```
# [20, 30, 40] [10, 20] [50]
```

Se vi aspettavate quattro elementi da numeri[1:4], benvenuti nel club.

8. ORDINARE

Uno modifica, l'altro restituisce

8.1 sort() contro sorted(): sul posto o lista nuova

Forma	Che cosa fa	Restituisce	Quando usarla
<code>lista.sort()</code>	riordina la lista sul posto	None	quando l'ordine originale non serve più
<code>sorted(lista)</code>	costruisce una lista nuova ordinata	la lista nuova	quando l'ordine originale serve ancora

Ordinare una lista si può fare in due modi, e la differenza è la stessa del capitolo 4: uno modifica, l'altro restituisce. `sorted()` è una FUNZIONE, `sort()` è un METODO della lista.

8.1 sorted() non tocca niente, sort() sì

```
zone = ["SUD", "NORD", "ISOLE", "CENTRO"]
```

```
# sorted() e' una FUNZIONE: costruisce una lista nuova ordinata e
# restituisce quella. L'originale non lo tocca, e la riga sotto lo dimostra.
print("Copia ordinata:", sorted(zone))
print("Originale:      ", zone)
```

```
# sort() e' un METODO della lista: riordina dentro l'oggetto e restituisce
# None. Dopo questa riga l'ordine di partenza non esiste piu' da nessuna
# parte: se vi serviva ancora, dovevate usare sorted().
zone.sort()
print("Dopo sort():   ", zone)
```

```
# Copia ordinata: ['CENTRO', 'ISOLE', 'NORD', 'SUD']
# Originale:      ['SUD', 'NORD', 'ISOLE', 'CENTRO']
# Dopo sort():    ['CENTRO', 'ISOLE', 'NORD', 'SUD']
```

8.2 lista = lista.sort(): perché la lista sparisce

```
zone = ["SUD", "NORD", "ISOLE"]

# sort() ordina e restituisce None. L'uguale prende quel None e lo mette
# sotto il nome zone: la lista ordinata e' esistita per un istante, e adesso
# non la indica piu' nessun nome.

zone = zone.sort()

# TRABOCCHETTO: "sort ordina, sorted restituisce".

#   zone.sort()           corretto, e non si assegna
#   zone = sorted(zone)   corretto, e si assegna
#   zone = zone.sort()    sbagliato, e qui non da' nessun errore

print(zone)
```

Stessa trappola di append, conseguenze peggiori: qui non avete perso un'aggiunta, avete perso la lista intera. Se il metodo modifica, si scrive da solo su una riga; se costruisce qualcosa di nuovo, si assegna.

```
# None
```

8.3 Ordine decrescente: reverse() e l'argomento reverse=True

```
pezzi = [130, 95, 140, 85, 160, 110]

# Due gesti separati, e si leggono come due gesti: prima metti in ordine,
# poi gira la lista. reverse() NON ordina: rovescia e basta.
pezzi.sort()
pezzi.reverse()

# TRABOCCHETTO: reverse() da solo, su una lista non ordinata, restituisce
# l'ordine di partenza al contrario. Non e' l'ordine decrescente.
print("Dal più grande:", pezzi)

# Dal più grande: [160, 140, 130, 110, 95, 85]
```

Esiste anche la forma in una riga sola, pezzi.sort(reverse=True), con l'argomento nominato di ieri. Nelle soluzioni trovate quella in due passi: si legge come due gesti.

8.4 Ordinare per un criterio: la lista di tuple

```
# La coppia si scrive con il CRITERIO davanti e l'etichetta dietro: qui il
# punteggio prima, il nome dopo. La lista contiene tuple, non stringhe.
coppie = [(78, "Rossi Anna"), (91, "Verdi Luca"), (55, "Bianchi Marco")]

# sort() su una lista di tuple confronta il PRIMO campo di ciascuna; solo a
# parita' di primo campo passa al secondo. Nessuna funzione da scrivere.
coppie.sort()
coppie.reverse()

# TRABOCCHETTO: scambiate i due campi e cambiate il criterio senza
# accorgervene: con ("Rossi Anna", 78) l'ordinamento diventa alfabetico.
print(coppie)

# [(91, 'Verdi Luca'), (78, 'Rossi Anna'), (55, 'Bianchi Marco')]
```

8.4 A parità di primo campo: il criterio secondario

```
coppie = [(3, "carta"), (8, "ordine"), (3, "chiedo")]

# Due coppie hanno 3 come primo campo: il confronto passa al secondo, e
# "carta" viene prima di "chiedo" in ordine alfabetico.

coppie.sort()

print("Crescente:", coppie)

# TRABOCCHETTO: reverse() rovescia TUTTO, anche il criterio secondario. Fra
# i due pari merito, a questo punto, "chiedo" sta prima di "carta": non e'
# un errore, ed e' quello che vedrete uscire dalle classifiche di oggi.

coppie.reverse()

print("Decrescente:", coppie)

# Crescente: [(3, 'carta'), (3, 'chiedo'), (8, 'ordine')]
# Decrescente: [(8, 'ordine'), (3, 'chiedo'), (3, 'carta')]
```

9. CONTI SU UNA LISTA

Quattro funzioni, una divisione e un denominatore a zero

9.1 len(), sum(), min(), max() e la media calcolata a mano

```
PEZZI = [130, 95, 140, 85, 160, 110]

# Quattro FUNZIONI INCORPORATE: non sono metodi della lista, non si scrivono
# con il punto. Prendono la collezione e restituiscono un numero solo,
# senza modificare niente.
print("Quanti mesi:", len(PEZZI))
print("Totale:", sum(PEZZI))
print("Minimo:", min(PEZZI))
print("Massimo:", max(PEZZI))

# TRABOCCHETTO: in Python di base non esiste media(). Si scrive a mano, ed
# e' un bene: chi scrive la divisione vede il DENOMINATORE, che e' l'unico
# punto in cui questa riga puo' esplodere (9.3). Il risultato e' un float
# anche quando la divisione e' esatta: 120.0.
print("Media:", sum(PEZZI) / len(PEZZI))

# Quanti mesi: 6      Totale: 720      Minimo: 85      Massimo: 160
# Media: 120.0
```

9.2 min() dà il valore, non l'elemento: il nome via index()

```
MESI = ["Aprile", "Maggio", "Giugno", "Luglio", "Agosto", "Settembre"]
PEZZI = [130, 95, 140, 85, 160, 110]

# Tre passi, in quest'ordine: il VALORE minimo, la sua POSIZIONE nella lista
# dei numeri, e la stessa posizione letta nella lista parallela dei nomi.
peggiore = min(PEZZI)
posizione = PEZZI.index(peggiore)
print(f"Mese peggiore: {MESI[posizione]} con {peggiore} pezzi")

# TRABOCCHETTO: la scorciatoia min(MESI) non dà errore e non dà il mese
# peggiore: ordina i NOMI e restituisce "Agosto", primo in alfabeto. Una
# risposta plausibile e sbagliata e' peggio di un traceback.
# TRABOCCHETTO: se il minimo compare due volte, index() dà la prima
# posizione e il secondo pari merito sparisce dal report.

# Mese peggiore: Luglio con 85 pezzi
```

9.3 La lista vuota: ZeroDivisionError sulla media

```
vuota = []
print("Somma:", sum(vuota))
print("Quanti:", len(vuota))
media = sum(vuota) / len(vuota)

Somma: 0
Quanti: 0
Traceback (most recent call last):
  File "prova.py", line 4, in <module>
    media = sum(vuota) / len(vuota)
            ~~~~~^~~~~~
ZeroDivisionError: division by zero
```

sum() e len() su una lista vuota rispondono 0 tutte e due, ed e' la risposta giusta: zero elementi, somma zero. Solo la riga 4 ha un problema. TRABOCCHETTO: la lista vuota non e' la causa, e' l'occasione; la causa e' il DENOMINATORE a zero. E' il primo errore della giornata che dipende dai dati arrivati e non da come avete scritto il codice.

9.3 La guardia: un if prima della divisione

```
vuota = []

# La GUARDIA: si controlla il denominatore prima di usarlo. Il ramo che dice
# di no viene per primo, così la divisione sta dove è già dimostrato che
# len() non vale zero.
if len(vuota) == 0:
    print("[!] Nessun dato: la media non si può calcolare")
else:
    print(f"Media: {sum(vuota) / len(vuota):.2f}")

# TRABOCCHETTO: il messaggio dice "non si può calcolare", non stampa 0.00.
# Una media zero e una media assente sono due informazioni diverse, e chi
# legge il report non ha modo di distinguerle.

# [!] Nessun dato: la media non si può calcolare
```

10. IL DIZIONARIO

Quando la posizione non serve: si chiede per nome

10.1 Accesso per chiave: il dizionario {chiave: valore}

```
# Graffe, e ogni elemento e' una COPPIA chiave: valore separata dai due
# punti. L'accoppiamento e' scritto nel codice, non affidato alla posizione.
LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}

# Le quadre ci sono ancora, ma dentro non c'e' un numero: c'e' la CHIAVE.
# Non e' una ricerca che scorre il dizionario, e' un accesso diretto: con
# quattro zone o con quarantamila costa uguale.
print(LISTINO["SUD"])

# TRABOCCHETTO: len() su un dizionario conta le COPPIE, cioe' le chiavi.
# Non somma i valori: risponde 4, non 39.0.
print(len(LISTINO), "zone a listino")

# 9.5
# 4 zone a listino
```

10.2 Le chiavi: tipi ammessi, unicità, maiuscole e minuscole

Domanda	Risposta
Che cosa può essere una chiave?	Una stringa o un numero. Nel corso useremo sempre stringhe
Due chiavi uguali?	Impossibile: la seconda sovrascrive la prima, in silenzio
"SUD" e "sud" sono la stessa chiave?	No. Sono diverse come due nomi diversi in rubrica
L'ordine delle chiavi conta?	Per trovare un valore no. Per stamparle sì: vedi 11.4
Un valore può ripetersi?	Sì, quanto volete. Due zone possono costare uguale

La terza riga e' quella che vi costera' tempo: {"NORD": 8.50, "nord": 99.00} e' un dizionario con DUE zone, e nessuno vi avvisa. Per questo l'input dell'utente si normalizza sempre prima di usarlo come chiave.

10.3 KeyError: la chiave assente nell'accesso diretto

```
LISTINO = {"NORD": 8.50, "CENTRO": 7.00}  
print(LISTINO["ESTERO"])
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    print(LISTINO["ESTERO"])  
~~~~~  
KeyError: 'ESTERO'
```

L'accesso diretto con le quadre e' INTRANSIGENTE: se la chiave non c'e', solleva KeyError. Non restituisce None, non restituisce zero, non la crea. TRABOCCHETTO: KeyError e IndexError sono lo stesso errore in due vestiti, ma si curano in modo diverso -- per l'indice si controlla len(), per la chiave si usa .get() oppure in. Va provocato PRIMA di insegnare .get().

11. LAVORARE CON I DIZIONARI

`.get()`, `.items()`, e l'ordine che non è alfabetico

11.1 .get(chiave, ripiego): l'accesso che non solleva KeyError

```
LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50}

# Chiave presente: .get() si comporta come le quadre e restituisce 9.5.
print(LISTINO.get("SUD", 0.0))

# Chiave assente: niente KeyError, niente interruzione. Restituisce il
# RIPIEGO, cioè il secondo argomento, e il programma tira dritto.
print(LISTINO.get("ESTERO", 0.0))

# Il ripiego lo scegliete voi e non deve essere un numero: qui è una
# stringa, perché uno 0.0 sembrerebbe un prezzo di favore.
print(LISTINO.get("ESTERO", "da concordare"))

# TRABOCCHETTO: in su un dizionario guarda le CHIAVI, mai i valori.
# "NORD" in LISTINO -> True   è una chiave
# 8.50 in LISTINO   -> False  è un valore, e in non lo vede
print("NORD" in LISTINO, "ESTERO" in LISTINO, 8.50 in LISTINO)

# 9.5
# 0.0
# da concordare
# True False False
```

11.1 Quadre o .get(): quale forma, e perché

Situazione	Forma da usare	Perché
La chiave DEVE esserci	<code>d["SUD"]</code>	se non c'è, i dati sono rotti e volete saperlo subito
La chiave PUÒ mancare per motivi legittimi	<code>d.get("ESTERO", ...)</code>	non è un errore, è un caso previsto
State contando occorrenze	<code>d.get(k, 0)</code>	la prima volta il conto parte da zero

Esiste uno strumento apposito per raccogliere l'errore dopo che e' successo, invece di prevenirlo: si chiama try/except, e non e' materia di queste undici giornate. Qui ci fermiamo al controllo preventivo, che resta la prima linea di difesa e resta quello giusto quando la chiave puo' mancare per un motivo previsto.

11.2 Assegnazione a chiave: inserimento e sovrascrittura

```
listino = {"NORD": 8.50, "CENTRO": 7.00}

listino["ESTERO"] = 22.00          # chiave nuova: aggiunge
listino["NORD"] = 8.90             # chiave esistente: sovrascrive

# La SINTASSI E' LA STESSA per le due operazioni, e il dizionario decide da
# solo guardando se la chiave c'e' gia'. Su una lista non funziona cosi':
# lista[7] = x su una lista corta e' IndexError, perche' una lista non crea
# posizioni che non esistono, un dizionario si'.

# TRABOCCHETTO: per SAPERE quale delle due e' successa serve un if ... in
#   prima. Senza, un report che dice "quattro zone aggiornate" nasconde due
#   zone aggiunte e due modificate, che per l'ufficio sono cose diverse.

print(listino)

# {'NORD': 8.9, 'CENTRO': 7.0, 'ESTERO': 22.0}
```

11.3 Scorrere: `.keys()`, `.values()`, `.items()`

Metodo	Vi dà	Uso tipico
<code>.keys()</code>	i nomi	elencare le zone, ordinarle
<code>.values()</code>	i valori	sum, min, max sui prezzi
<code>.items()</code>	le coppie	stampare la tabella, aggregare

Un dizionario si guarda in tre modi, secondo quello che vi serve: solo i nomi, solo i valori, oppure le due cose appaiate. Con `.items()` servono due nomi a sinistra del `in`, perché restituisce coppie: è lo stesso unpacking che usate dal Giorno 03 con `.split(";")`.

11.3 .values() per i conti, .items() per la tabella

```
LISTINO = {"NORD": 8.50, "CENTRO": 7.00, "SUD": 9.50, "ISOLE": 14.00}

# .values() consegna i soli VALORI, ed e' cio' che rende usabili le funzioni
# del capitolo 9: sum(), min() e max() vogliono una collezione di numeri, e
# un dizionario cosi' com'e' non lo e'.
print("Somma dei prezzi:", sum(LISTINO.values()))
print("Il più caro:", max(LISTINO.values()))

# TRABOCCHETTO: sum(LISTINO) senza .values() prova a sommare le CHIAVI, che
# qui sono stringhe, e si ferma con TypeError.

# .items() consegna a ogni giro una COPPIA (chiave, valore), e i due nomi a
# sinistra del in la aprono: e' lo stesso unpacking del Giorno 03.
for zona, prezzo in LISTINO.items():
    print(f" {zona:<7} {prezzo:>5.2f} euro")

# Somma dei prezzi: 39.0          Il più caro: 14.0
#   NORD      8.50 euro          SUD      9.50 euro
#   CENTRO    7.00 euro          ISOLE    14.00 euro
```

11.4 L'ordine delle chiavi è quello di inserimento, non l'alfabeto

Da Python 3.7 in poi, un dizionario ricorda l'ordine in cui le chiavi sono state inserite. Non le ordina alfabeticamente. Non le ordina per valore. Le ridà come le ha ricevute.

È la cosa che oggi si sbaglia di più nel prevedere l'output:
metà aula, davanti a un dizionario, si aspetta l'ordine alfabetico.
Va fatto sbagliare adesso, non dentro il mini-progetto del pomeriggio.

Conseguenza pratica per i vostri report

l'ordine di uscita è una decisione, e va presa apposta
l'alfabeto si chiede: `sorted()` sulle chiavi restituisce una lista ordinata
un report che cambia ordine da un mese all'altro perché sono cambiati i dati
in ingresso è un report che nessuno confronta più con quello di prima

11.4 Inserimento contro alfabeto, a confronto

```
# Chiavi inserite fuori ordine apposta, e CENTRO arriva per ultimo: quindi

# uscira' per ultimo.

listino = {"SUD": 9.50, "NORD": 8.50}

listino["CENTRO"] = 7.00


# Scorrendo le chiavi si ottiene l'ORDINE DI INSERIMENTO, tale e quale.
for zona in listino.keys():

    print(zona)


# L'alfabeto va CHIESTO: sorted() costruisce una lista nuova ordinata e
# ", ".join(...) la unisce in una riga sola.

# TRABOCCHETTO: sorted() qui non riordina il dizionario, che resta com'era:
#     quello che ordinate e' una lista di chiavi costruita a parte.
print(", ".join(sorted(listino.keys()))


# SUD

# NORD

# CENTRO          <- ordine di INSERIMENTO

# CENTRO, NORD, SUD  <- ordine ALFABETICO, chiesto apposta
```

12. CONTARE LE OCCORRENZE

Le quattro parole di codice che userete per sempre

12.1 Il conteggio per categoria: perché i contatori a mano non bastano

Quanti ticket per categoria. Quante spedizioni per zona.

Quante volte compare ogni parola. Quanti pazienti per colore di triage.

È la stessa domanda vestita in cinque modi.

La strada che viene istintiva

una variabile per categoria e una catena di elif: quattro contatori, dodici righe

Funziona, e ha un difetto grosso:

DOVETE SAPERE IN ANTICIPO QUALI SONO LE CATEGORIE.

Il giorno che l'ospedale introduce il codice ARANCIONE, quel programma lo

ignora in silenzio. Non dà errore: dà un report che non lo nomina.

12.2 conteggi[k] = conteggi.get(k, 0) + 1: il conta-occorrenze

```
triage = ["VERDE", "GIALLO", "VERDE", "ROSSO", "VERDE", "BIANCO", "GIALLO"]

# Il dizionario parte VUOTO: nessuna categoria dichiarata in anticipo, ed e'
# questa riga a rendere il conteggio indipendente dai dati.
conteggi = {}

for colore in triage:
    # Tre gesti in una riga sola, da destra a sinistra:
    #   .get(colore, 0)  quanto valeva finora? se e' la prima volta, zero
    #   + 1              uno in piu'
    #   conteggi[...] = rimetti al suo posto, creando la chiave se non
    #                   c'era (e' l'inserimento di 11.2)
    conteggi[colore] = conteggi.get(colore, 0) + 1

# TRABOCCHETTO: senza il ripiego 0, la prima volta .get() restituisce None e
#   None + 1 si ferma con TypeError. Quello zero non e' decorazione: e' il
#   caso "prima volta" trattato senza un if apposta.
print(conteggi)

# {'VERDE': 3, 'GIALLO': 2, 'ROSSO': 1, 'BIANCO': 1}
```

12.2 La riga della giornata, pezzo per pezzo

Pezzo	Che cosa fa
<code>conteggi.get(colore, 0)</code>	quanto valeva finora? Se è la prima volta, zero
<code>+ 1</code>	uno in più
<code>conteggi[colore] = ...</code>	rimetti il risultato al suo posto, creando la chiave se non c'era

Analogia: le tacche sul muro. Ogni volta che passa qualcuno fate una tacca accanto al suo nome; se il nome non c'era ancora, lo scrivete e cominciate da zero. Quel "se non c'era, zero" e' tutto `.get(chiave, 0)`: niente elenco dei nomi appeso prima, il muro si riempie da solo.

12.3 Stampare i conteggi: quota percentuale e barra di cancelletti

```
conteggi = {"VERDE": 3, "GIALLO": 2, "ROSSO": 1, "BIANCO": 1}

# Il denominatore si calcola UNA VOLTA SOLA, fuori dal ciclo: e' il numero
# di casi contati in tutto, non il numero di categorie.
totale = sum(conteggi.values())

for colore, quanti in conteggi.items():

    # quota e' una frazione fra 0 e 1, non una percentuale: 3 / 7 fa
    # 0.4285... ed e' lo specificatore >5.1% a moltiplicare per cento e ad
    # aggiungere il segno. Moltiplicare anche voi darebbe 4285.7%.
    quota = quanti / totale

    # "#" * quanti ripete la stringa: e' la moltiplicazione del Giorno 02,
    # e fa da grafico senza nessuna libreria.

    print(f" {colore:<8} {quanti} {quota:>5.1%} {'#' * quanti}")

# TRABOCCHETTO: len(conteggi) vale 4 (le categorie) e sum(.values()) vale 7
# (i casi). Usare il primo come denominatore da' percentuali che sommano
# a 175%, e nessuno se ne accorge finche' non le somma.

# VERDE    3  42.9% ###          ROSSO    1  14.3%  #
# GIALLO   2  28.6% ##          BIANCO    1  14.3%  #
```

12.4 Aggregare per chiave: + importo al posto di + 1

```
# Dati in ingresso: una lista di TUPLE (categoria, importo), che e' il
# formato del capitolo 14.

ORDINI = [("AUDIO", 99.80), ("VIDEO", 219.00), ("AUDIO", 49.90),
          ("ACCESSORI", 27.00), ("VIDEO", 59.90)]

# AGGREGAZIONE: la riga e' quella di 12.2 con + importo al posto di + 1, e
# da "quante volte compare" diventa "quanto vale in tutto".

totali = {}

for categoria, importo in ORDINI:
    totali[categoria] = totali.get(categoria, 0) + importo

# Il secondo ciclo stampa: prima si aggrega, poi si formatta. Due cicli
# separati perche' sono due mestieri, non per distrazione.

for categoria, importo in totali.items():
    print(f" {categoria:<11} {importo:>7.2f} euro")

# TRABOCCHETTO: il ripiego resta 0, un int, mentre gli importi sono float.
# Funziona perche' int + float da' float; con "0" fra virgolette la prima
# somma si fermerebbe con TypeError.

# AUDIO          149.70 euro
# VIDEO          278.90 euro
# ACCESSORI      27.00 euro
```

12.5 Categorie dichiarate contro categorie ricavate dai dati

	Categorie dichiarate	Categorie ricavate
Dove sta l'elenco	nel codice, scritto a mano	nei dati, e il codice lo scopre
Categoria nuova	va aggiunto un if	compare da sola
Categoria sparita	resta a zero per sempre	non compare più
Costo di una modifica	una riga per categoria	nessuna

Nel Giorno 04 avevate contato sette parole spia scelte da voi, una per if: l'elenco stava nel codice. Il prezzo c'e' e va detto: con le categorie ricavate non sapete in anticipo quali chiavi troverete, quindi il report va sempre ordinato apposta (11.4) e un refuso nei dati diventa una categoria a se'. E' il salto dell'esercizio 07.5.

13. LA TUPLA

Come una lista, ma non si tocca

13.1 La tupla (...): in lettura si comporta come una lista

```
# Una tupla: quattro campi DIVERSI di una cosa sola. Non e' un elenco di
# cose omogenee, e' un RECORD, la riga di un registro. Guardate i tipi
# mescolati -- str, int, int, str: in una lista sarebbero un campanello
# d'allarme, in una tupla sono la norma.
riga = ("Neri Sara", 2, 64, "NON IDONEO")

# In lettura la tupla si comporta esattamente come una lista.
print(riga)

print("Campi:", len(riga))

print("Nome:", riga[0])

print("Esito:", riga[-1])

# TRABOCCHETTO: stampata cosi' esce con le tonde e gli apici. E' il modo in
# cui Python la mostra a voi, non un formato da consegnare: per un report
# si scorre e si formatta, come al capitolo 14.

# ('Neri Sara', 2, 64, 'NON IDONEO')
# Campi: 4
# Nome: Neri Sara
# Esito: NON IDONEO
```

13.1 Immutabilità e TypeError: non c'è niente da aggirare

```
riga = ("Neri Sara", 2, 64, "NON IDONEO")  
riga[2] = 74
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    riga[2] = 74  
    ~~~^^^  
TypeError: 'tuple' object does not support item assignment
```

La tupla e' IMMUTABILE: niente assegnazione per posizione, niente append, remove, sort. L'errore non e' KeyError ne' IndexError -- la posizione 2 esiste eccome -- ma TypeError: e' il TIPO a non prevedere l'operazione. TRABOCCHETTO: se quel campo deve cambiare, la scelta della tupla era sbagliata; se non deve cambiare, questo traceback e' il servizio per cui l'avete scelta. Analogia: la lista e' una lavagnetta, la tupla e' un cartellino scritto a penna e inchiodato al muro.

13.2 Correggere un record: la tupla si sostituisce, non si ritocca

```
# La LISTA e' mutabile, la TUPLA che contiene no: correggere un dato vuol
# dire costruire la tupla nuova e metterla al posto di quella vecchia, con
# l'assegnazione per posizione del 3.3.

candidature = [("Neri Sara", 2, 64, "NON IDONEO")]

candidature[0] = ("Neri Sara", 2, 74, "IDONEO")


# TRABOCCHETTO: qui cambiano DUE campi, punteggio ed esito. Con una lista si
#   ritoccherebbero uno alla volta, e fra i due ritocchi il dato sarebbe
#   incoerente: 74 punti ancora marcati NON IDONEO. La tupla non concede
#   quel mezzo passaggio: o la riga vecchia o la nuova.

print(candidature[0])


# ('Neri Sara', 2, 74, 'IDONEO')
```

13.3 Unpacking: tanti nomi a sinistra quanti sono i campi

```
riga = ("Neri Sara", 2, 64, "NON IDONEO")

# UNPACKING: quattro nomi a sinistra, quattro campi a destra, accoppiati per
# posizione. Da qui in poi non si scrive piu' riga[2] ma punteggio, ed e' la
# riga che rende leggibile tutto il capitolo 14.
nome, anni, punteggio, esito = riga
print(f"{nome}: {punteggio} punti, esito {esito}")

# TRABOCCHETTO: i nomi devono essere ESATTAMENTE quanti i campi. Con tre
# nomi Python si ferma con "too many values to unpack" e indica la riga
# dell'unpacking, non la riga dei dati che ne ha uno di troppo: quando le
# tabelle arriveranno da un file vi fara' cercare nel posto sbagliato.

# Neri Sara: 64 punti, esito NON IDONEO
```

13.4 Criteri di scelta fra tupla e lista

Domanda	Se la risposta è sì
I valori sono dello stesso tipo e possono essere tanti?	lista (sei prodotti, dodici mesi)
I valori sono campi diversi di una stessa cosa?	tupla (nome, anni, punteggio, esito)
Devo aggiungere e togliere elementi?	lista
Deve restare com'è, e se cambia è un'altra cosa?	tupla
Mi serve come riga di una tabella?	tupla, dentro una lista

Nella pratica: la lista è il registro, la tupla è la riga del registro. E il registro è il capitolo che segue.

14. UNA TABELLA IN PYTHON

Lista di tuple: ogni tupla una riga, ogni posizione una colonna

14.1 Lista di tuple: righe e colonne

```
# LISTA DI TUPLE: la lista fa da registro, ogni tupla e' una riga, ogni
# posizione dentro la tupla e' una colonna. L'ordine dei campi e' un
# contratto: vale per tutte le righe, e non si cambia per una sola.

CANDIDATURE = [

    ("Rossi Anna", 3, 78, "IDONEO"),

    ("Bianchi Marco", 1, 55, "NON IDONEO"),

    ("Verdi Luca", 6, 91, "IDONEO"),

]

# L'intestazione si stampa a parte, con le STESSE larghezze delle righe: e'
# cio' che tiene incolonnato il report.

print(f"{'NOME':<15}{'ANNI':>4}{'TEST':>6}  ESITO")

# L'unpacking dentro il for da' un NOME a ogni colonna. Con riga[0], riga[1],
# riga[2] girerebbe uguale e fra un mese non si leggerebbe piu'.

for nome, anni, punteggio, esito in CANDIDATURE:

    print(f"{'nome':<15}{'anni':>4}{'punteggio':>6}  {esito}")

# NOME          ANNI  TEST  ESITO
# Rossi Anna      3    78  IDONEO
# Bianchi Marco   1    55  NON IDONEO
# Verdi Luca      6    91  IDONEO
```

14.2 Contare e sommare su una tabella: un giro, due accumulatori

```
CANDIDATURE = [("Rossi Anna", 3, 78, "IDONEO"),
                ("Bianchi Marco", 1, 55, "NON IDONEO"),
                ("Verdi Luca", 6, 91, "IDONEO")]

# Due accumulatori azzerati fuori: uno conta le righe che passano il filtro,
# l'altro somma una colonna.

idonei = 0
somma_test = 0

for nome, anni, punteggio, esito in CANDIDATURE:
    # Fuori dall'if riguarda TUTTE le righe; dentro l'if, solo quelle che
    # soddisfano la condizione. E' la sola differenza fra i due.
    somma_test += punteggio
    if esito == "IDONEO":
        idonei += 1

print(f"Idonei: {idonei} su {len(CANDIDATURE)}")

# TRABOCCHETTO: il denominatore qui e' len(CANDIDATURE), tutte le righe. Se
# volevate la media dei soli idonei, il denominatore era idonei, e la riga
# andrebbe protetta con un if: idonei puo' valere zero.
print(f"Media del test: {somma_test / len(CANDIDATURE):.2f}")

# Idonei: 2 su 3
# Media del test: 74.67
```

14.3 Dalla tabella alla classifica: coppie, sort, reverse, fetta

```
CANDIDATURE = [("Rossi Anna", 3, 78, "IDONEO"),
                ("Bianchi Marco", 1, 55, "NON IDONEO"),
                ("Verdi Luca", 6, 91, "IDONEO")]

# 1. PROIEZIONE: dalla tabella a quattro campi si ricava una lista di coppie
# (criterio, etichetta). Il punteggio davanti: e' la regola di 8.4.
coppie = []

for nome, anni, punteggio, esito in CANDIDATURE:
    coppie.append((punteggio, nome))

# 2. ORDINAMENTO: crescente, poi rovesciato. Nessun criterio da scrivere.
coppie.sort()
coppie.reverse()

# 3. TAGLIO: coppie[0:2] sono i primi due. Se la lista ne avesse uno solo la
# fetta ne darebbe uno, senza errore: lo slicing non si lamenta.
posizione = 0

for punteggio, nome in coppie[0:2]:
    posizione += 1
    print(f" {posizione}. {nome:<15}{punteggio:>3}")

# 1. Verdi Luca      91
# 2. Rossi Anna      78
```

14.4 Lo stesso codice quando la tabella arriverà da un file

Più avanti, quando impareremo a leggere un file di testo

la lista di tuple non la scriverete più a mano: arriverà dal file,

una riga per tupla, con i campi separati dal punto e virgola

il resto del programma resterà identico: cambieranno solo le prime righe,

quelle che caricano i dati: è il motivo per cui oggi lo usiamo più di tutti

E dopo? Più avanti nel corso

incontrerete strumenti che trattano una tabella come un oggetto solo

e fanno queste stesse cose su milioni di righe

le liste di oggi sono esattamente ciò che quegli strumenti sostituiscono

Impararle adesso è la ragione per cui, quando quella riga vi restituirà

un numero strano, saprete dove guardare.

15. SCEGLIERE LA STRUTTURA GIUSTA

Se ricordate una tabella sola di oggi, ricordate questa

15.1 Tabella di decisione: dal dato in ingresso alla struttura

Che cosa avete in mano	Struttura	Esempio
Un elenco di cose dello stesso genere, in un ordine che conta	lista	la coda d'attesa, i mesi, i prodotti
Coppie nome-valore, dove cercate per nome	dizionario	il listino per zona, la rubrica
I campi fissi di una singola cosa	tupla	una candidatura, un ordine
Molte cose con gli stessi campi fissi	lista di tuple	il registro delle candidature
Quante volte compare ciascuna cosa	dizionario di conteggi	ticket per categoria, parole
Un totale per ciascuna cosa	dizionario di somme	fatturato per categoria

Le ultime due righe sono lo stesso oggetto con dentro numeri diversi, e si costruiscono con la stessa riga di codice: se vi ricordate una cosa sola di tutta la giornata, ricordatevi quella.

15.2 Tre casi d'ufficio da classificare

"I colori del triage ammessi sono quattro."

Elenco fisso e ordinato per gravità: LISTA. Se poi serve sapere quanti pazienti per colore, quello è un DIZIONARIO DI CONTEGGI, ed è un'altra cosa.

"A ogni fascia ISEE corrisponde una tariffa."

DIZIONARIO. Con due liste parallele funzionerebbe, e il primo che inserisce una fascia in mezzo rompe tutto.

"Ogni spedizione ha un codice, una zona e un peso, e ce ne sono venti."

Ogni spedizione è una TUPLA di tre campi, le venti sono una LISTA:
è già il formato con cui quei dati arriveranno il giorno che li leggerete da un file, e non ci sarà niente da riscrivere.

L'errore più comune: mettere in una lista cose che hanno un nome.

16. ERRORI TIPICI DELLA GIORNATA

Tre danno un traceback. Gli altri quattro no, e sono i peggiori

E1 — IndexError: la posizione che non esiste

```
zone = ["NORD", "CENTRO", "SUD"]  
print(zone[3])
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    print(zone[3])  
      ~~~~^^^  
IndexError: list index out of range
```

Tre elementi, indici 0, 1, 2: l'indice 3 non esiste e Python non lo inventa. "out of range" vuol dire fuori dall'intervallo valido. TRABOCCHETTO: la cura non e' scrivere [2], e' capire perche' il programma chiedeva la posizione 3. Quasi sempre c'e' un ciclo che fa un giro di troppo, o un + 1 su un indice che partiva gia' da zero.

E2 — KeyError: la chiave che non c'è

```
listino = {"NORD": 8.50, "CENTRO": 7.00}  
print(listino["ESTERO"])
```

```
Traceback (most recent call last):  
  File "prova.py", line 2, in <module>  
    print(listino["ESTERO"])  
~~~~~  
KeyError: 'ESTERO'
```

KeyError nomina fra apici la chiave che mancava: e' l'informazione piu' utile del traceback, perche' spesso la chiave c'e' ma scritta diversa. TRABOCCHETTO: sospettate delle maiuscole e degli spazi ai bordi -- "estero", "ESTERO " e "ESTERO" sono tre chiavi diverse. La cura e' .get(chiave, ripiego) quando la chiave puo' mancare, e un if ... in ... quando volete decidere voi.

E3 — ValueError: togliere o cercare ciò che non c'è

```
zone = ["NORD", "CENTRO"]  
zone.remove("ISOLE")
```

```
ValueError: list.remove(x): x not in list
```

Vale identico per `.index()`, e per l'unpacking di una tupla con un numero di campi sbagliato:

```
too many values to unpack
```

ValueError: l'operazione esiste e il TIPO va bene, ma il VALORE passato non e' utilizzabile qui. TRABOCCHETTO: la cura e' il controllo prima, if "ISOLE" in zone, cioe' la funzione `posizione_o_zero` del capitolo 5. Su una lista non esiste un `.remove()` con ripiego, come invece `.get()` lo e' per il dizionario.

E4 — sort() che restituisce None

```
zone = ["SUD", "NORD", "ISOLE"]  
  
zone = zone.sort()  
  
# Nessun errore, e la lista non c'e' piu'. Vale identico per append,  
# insert, remove e reverse: tutti restituiscono None.  
# TRABOCCHETTO: l'errore vero arriva piu' tardi e altrove, quando qualcuno  
# fa len(zone) e legge un TypeError su NoneType. Il punto in cui il  
# programma si ferma non e' il punto del difetto.  
  
print(zone)  
  
# None
```

Frequentissimo e silenzioso. Due righe in diretta, e da ripetere: e' lo stesso di lista = lista.append(x) del capitolo 4.

E5 — Assegnazione creduta copia

```
originale = ["NORD", "CENTRO"]

# Un solo oggetto, due nomi. Nessuna copia e' stata fatta.
copia = originale
copia.append("SUD")

# TRABOCCHETTO: il danno non si vede qui. Si vede fra venti righe, quando
# qualcuno stampa i dati di partenza e li trova cambiati. Se volete due
# liste davvero separate: copia = originale[:].
print("originale:", originale)

# originale: ['NORD', 'CENTRO', 'SUD']
```

*Due etichette, un barattolo. Questo errore non si vede finche' qualcuno non si accorge che i dati di partenza sono cambiati, e a quel punto e' gia' in un report.
Domanda all'aula, e aspettate la risposta: perche' e' cambiata anche l'altra?*

E6 — Modificare una lista mentre la si scorre

```
numeri = [1, 2, 4, 5, 6]

# Il for avanza PER POSIZIONE anche quando non ve la mostra: remove()
# accorcia la lista sotto i piedi del ciclo, tutti scalano di un posto, e la
# posizione successiva salta un elemento.
for numero in numeri:
    if numero % 2 == 0:
        numeri.remove(numero)

# TRABOCCHETTO: il 4 sopravvive. Nessun errore, nessun avviso, e su una
# lista lunga il risultato e' sbagliato di poco: il tipo di difetto che
# passa la revisione.
print(numeri)

# [1, 4, 5]
```

E6 — La cura: scorrere una copia

```
numeri = [1, 2, 4, 5, 6]

# numeri[:] e' una lista NUOVA, e il ciclo scorre quella: si accorcia
# l'originale mentre si legge la copia, che nessuno sta toccando.
for numero in numeri[:]:
    if numero % 2 == 0:
        numeri.remove(numero)

# TRABOCCHETTO: le due quadre vuote sono tutta la differenza fra [1, 4, 5]
# e [1, 5]. Sono anche il genere di dettaglio che si perde copiando il
# codice a mano dalla lavagna: guardatele bene adesso.
print(numeri)

# [1, 5]
```

E7 — len(), sum(.values()) e .get(): tre numeri diversi

```
conteggi = {"gratis": 3, "vinci": 2, "rimborso": 1}

# 1. len() -> quante CHIAVI diverse: tre parole distinte.
print(len(conteggi))

# 2. sum(.values()) -> quante OCCORRENZE in tutto: sei apparizioni.
print(sum(conteggi.values()))

# 3. .get(k, 0) -> quante volte UNA parola specifica: zero, non c'è.
print(conteggi.get("clicca", 0))

# TRABOCCHETTO: chiamare "totale" il primo dei tre e' l'errore che manda in
# riunione il numero sbagliato, e non e' un errore di Python. Il nome
# della variabile deve dire QUALE delle tre domande avete fatto.

# 3
# 6
# 0
```

RIEPILOGO DEL GIORNO

Sedici righe da riguardare stasera

Riepilogo del giorno (1/2)

Il numero di variabili si decide scrivendo il codice, il numero di dati arriva dopo.

Lista: [...], indice da 0, indice negativo dalla fine, indice massimo len meno uno.

append in fondo, insert in mezzo, remove per valore, pop per posizione e restituisce.

Chi modifica non si assegna: lista = lista.sort() distrugge la lista, in silenzio.

in prima di index, sempre. count è l'unico che su un valore assente risponde zero.

for elemento in lista quando la posizione non serve, enumerate(lista, 1) quando serve.

lista[a:b] con l'estremo destro escluso, come sulle stringhe. lista[:] è la copia vera.

sorted() costruisce una lista nuova, sort() modifica e restituisce None.

Per ordinare per un criterio: lista di tuple con il criterio in prima posizione.

sum, min, max, len e la media a mano. min dice quanto, non chi: il nome con .index().

Riepilogo del giorno (2/2)

Dizionario: coppie chiave-valore, si cerca per nome. Le chiavi distinguono le maiuscole.

.get(chiave, ripiego) quando la chiave può mancare, d[chiave] quando non può.

L'ordine delle chiavi è quello di inserimento, non alfabetico: l'alfabeto si chiede.

conteggi[k] = conteggi.get(k, 0) + 1. Con + importo al posto di + 1 è aggregazione.

Tupla: come una lista ma non si modifica. È la riga di dati, e si sostituisce intera.

Lista di tuple: la tabella. Unpacking nel for per dare un nome alle colonne.

Scegliere: elenco ordinato lista, nome-valore dizionario, campi fissi tupla,
molte righe con gli stessi campi lista di tuple.

Errori: IndexError, KeyError, ValueError, TypeError sulla tupla. E i silenziosi:

sort() che dà None, la copia che non copia, la lista modificata mentre la si scorre.

Grazie.

shaping the skills of tomorrow

